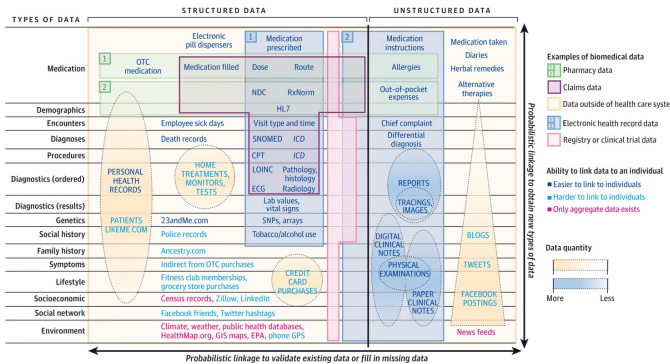
Large Language Models: The Basics

Alexander McLain

BIOS 735: Machine Learning for Public Health Applications

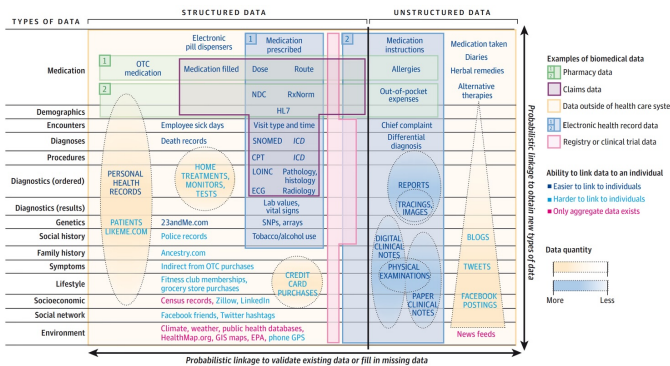
Why talk about Large language models (LLMs) in this class?

- LLMs like ChatGPT, Claude, Gemini, and Llama are now used for just about everything in the research pipeline.
- Much of public health data is **text**: clinical notes, death certificates, survey free-text, social media, etc.
- An LLM is "just" a very large neural network trained to do a **classification problem**.



Why talk about Large language models (LLMs) in this class?

- LLMs like ChatGPT, Claude, Gemini, and Llama are now used for just about everything in the research pipeline.
- Much of public health data is **text**: clinical notes, death certificates, survey free-text, social media, etc.
- An LLM is "just" a very large neural network trained to do a **classification problem**.
- **Goal for this week:** understand what is under the hood, so you can use these tools well and critique studies that use them.



What we already know that we will use

- **Neural networks:** layers, activation functions, weights learned by gradient descent (backpropagation).
- **Softmax and cross-entropy:** the output layer for multi-class classification,

$$P(Y = k \mid \boldsymbol{x}) = \frac{\exp(z_k)}{\sum_{j=1}^{K} \exp(z_j)}.$$

- **Dimension reduction:** representing a high-dimensional object with a short vector of numbers (PCA scores, UMAP coordinates).
- **Training/validation/test splits and overfitting.**

What we already know that we will use

- **Neural networks:** layers, activation functions, weights learned by gradient descent (backpropagation).
- **Softmax and cross-entropy:** the output layer for multi-class classification,

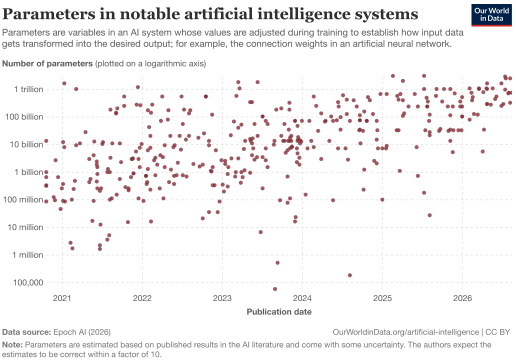
$$P(Y = k \mid \boldsymbol{x}) = \frac{\exp(z_k)}{\sum_{j=1}^{K} \exp(z_j)}.$$

- **Dimension reduction:** representing a high-dimensional object with a short vector of numbers (PCA scores, UMAP coordinates).
- **Training/validation/test splits and overfitting.**

An LLM puts these together: it represents words as vectors (like DR), passes them through a very deep neural network, and ends with a softmax over every possible next word.

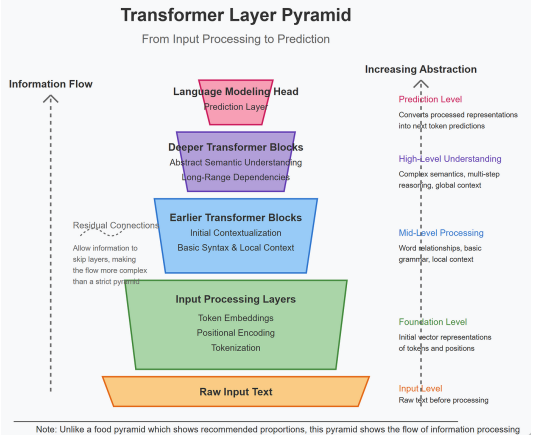
A short history

- **1950s–2000s:** rules; *n*-gram counts.
- **2013:** word2vec — words as dense vectors.
- **2014–2016:** recurrent neural networks (RNNs) and the first "attention".
- **2017:** the Transformer ("Attention Is All You Need").
- **2018–2020:** BERT, GPT-2, GPT-3 (175 billion parameters).
- **2022:** ChatGPT; instruction tuning + human feedback.
- **2023–now:** GPT-4, Claude, Gemini, Llama; open-weight and "reasoning" models; Med-PaLM.



The big picture

1. **Tokenization:** break text into pieces.
2. **Embeddings:** turn each piece into a vector of numbers.
3. **Transformer:** let the vectors "talk to each other" (attention) so each word's vector reflects its context.
4. **Prediction:** softmax over the vocabulary gives the probability of the next token.



What is a language model?

- A **language model** is a probability distribution over sequences of words (tokens) $w_1, w_2, \ldots, w_T$.
- By the chain rule of probability,

$$P(w_1, \ldots, w_T) = \prod_{t=1}^{T} P(w_t \mid w_1, \ldots, w_{t-1}).$$

- So we only need a model for the **next word given the previous words**.

What is a language model?

- A **language model** is a probability distribution over sequences of words (tokens) $w_1, w_2, \ldots, w_T$.
- By the chain rule of probability,

$$P(w_1, \ldots, w_T) = \prod_{t=1}^{T} P(w_t \mid w_1, \ldots, w_{t-1}).$$

- So we only need a model for the **next word given the previous words**.
- This is a classification problem!
  - Features: the previous words (the "context").
  - Outcome: the next word, one of K classes, where K is the size of the vocabulary (often 50,000–200,000).

Next-token prediction: an example

"The patient was diagnosed with type 2 _____"

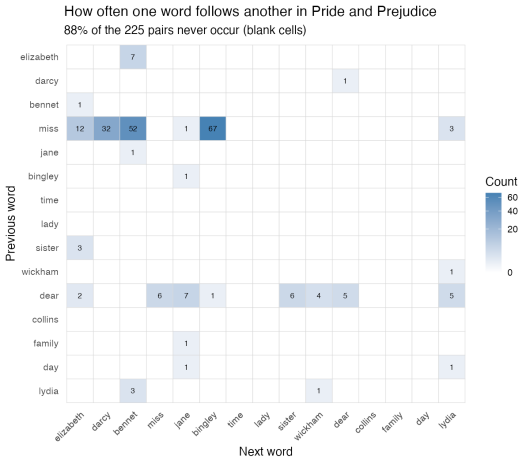
Next token	Probability
diabetes	0.91
diabetic	0.03
DM	0.02
myocardial	0.005
⋮	⋮

The simplest language model: n-grams

- Assume the next word depends only on the previous $n-1$ words.
- Bigram ($n = 2$) model:

$$\hat{P}(w_t \mid w_{t-1}) = \frac{\#(w_{t-1}, w_t)}{\#(w_{t-1})}.$$

- Just counting (the multinomial MLE).

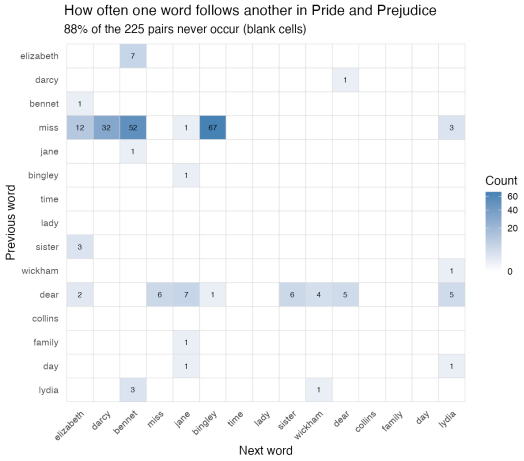


The simplest language model: n-grams

- Assume the next word depends only on the previous $n-1$ words.
- Bigram ($n = 2$) model:

$$\hat{P}(w_t \mid w_{t-1}) = \frac{\#(w_{t-1}, w_t)}{\#(w_{t-1})}.$$

- Just counting (the multinomial MLE).
- Problems:
  - Parameters grow like K^n; most counts are zero.
  - "Diabetes" and "diabetic" are unrelated.
  - Can't use context from 50 words back.



From *n*-grams to neural language models

- Neural language models fix the *n*-gram problems by
  1. representing words as **vectors** (so similar words share information), and
  2. using a **neural network** to combine the context into a prediction.
- The output layer is the softmax we saw in the neural network notes, but with $K \approx 100{,}000$ classes:

$$P(w_t = k \mid w_{<t}) = \frac{\exp(z_k)}{\sum_{j=1}^{K} \exp(z_j)}, \qquad \boldsymbol{z} = f_{\boldsymbol{\theta}}(w_1, \ldots, w_{t-1}).$$

- Training minimizes the **cross-entropy** (negative log-likelihood) of the observed next word:

$$L(\boldsymbol{\theta}) = -\sum_{t} \log P_{\boldsymbol{\theta}}(w_t \mid w_{<t}).$$

Tokenization

- Models see **tokens**, not words. Options:
  - **Characters:** small vocabulary, but very long sequences.
  - **Words:** huge vocabulary; new words ("COVID-19" in 2019) are unknown.
  - **Subwords** (what LLMs use): common words are one token, rare words are split into pieces.
- **Byte-pair encoding (BPE):** start with characters; repeatedly merge the most frequent adjacent pair.
- Rule of thumb: 1 token $\approx 3/4$ of an English word.



Why tokenization matters in practice

- **Cost and limits** are measured in tokens (price per million tokens; context window of X tokens).
- **Medical and non-English text** uses more tokens per word, so it is more expensive and fills the context window faster.
- Some well-known LLM failures come from tokenization:
  - Counting letters ("How many r's in strawberry?").
  - Arithmetic on long numbers (numbers are split into odd chunks).
  - Spelling and abbreviations ("q.i.d.", "SOB" = shortness of breath).

Why tokenization matters in practice

- **Cost and limits** are measured in tokens (price per million tokens; context window of X tokens).
- **Medical and non-English text** uses more tokens per word, so it is more expensive and fills the context window faster.
- Some well-known LLM failures come from tokenization:
  - Counting letters ("How many r's in strawberry?").
  - Arithmetic on long numbers (numbers are split into odd chunks).
  - Spelling and abbreviations ("q.i.d.", "SOB" = shortness of breath).

Discussion: You want to use an LLM to code 50,000 open-ended survey responses written in English and Spanish. What tokenization issues might come up?

Representing words as vectors

- **One-hot encoding:** word k is a vector of length K with a 1 in position k.
  - Every pair of words is equally far apart.
  - "fever" is as different from "pyrexia" as from "bicycle".
- **Embedding:** each token is mapped to a short, dense vector $\boldsymbol{e}_k \in \mathbb{R}^d$ (in LLMs, $d \approx 768$ to 12,000).
- The embedding is the first layer of the network: row k of a weight matrix $\boldsymbol{W}_E$, learned like any other weights.

One-hot ($K = 100{,}000$ columns)

	1	⋯	4,102	⋯	57,318
fever	0	⋯	1	⋯	0
pyrexia	0	⋯	0	⋯	1
bicycle	0	⋯	0	⋯	0

Embedding ($d = 4$; made-up numbers)

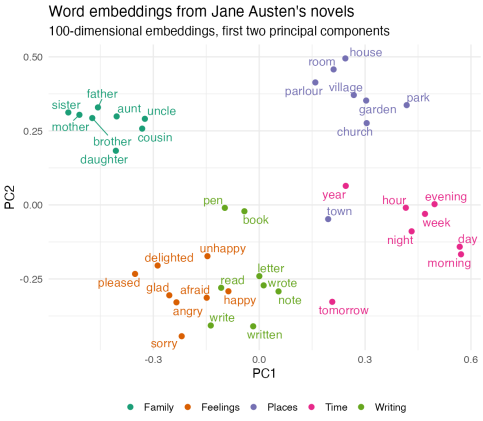
	e_1	e_2	e_3	e_4
fever	0.82	−0.11	0.45	0.30
pyrexia	0.79	−0.05	0.41	0.36
bicycle	−0.20	0.91	−0.33	0.02

fever and pyrexia now have similar vectors.

Embeddings capture meaning

- Words used in similar contexts get similar vectors.
- Plot: embeddings from Jane Austen's novels (counts of nearby words, then SVD). We build these in the class example.
- Similarity: $\cos(\boldsymbol{e}_a, \boldsymbol{e}_b) = \dfrac{\boldsymbol{e}_a^\top \boldsymbol{e}_b}{\|\boldsymbol{e}_a\| \, \|\boldsymbol{e}_b\|}$.

Word	Nearest neighbors (cosine)	
sister	brother (0.78)	mother (0.76)
morning	day (0.82)	evening (0.75)
letter	note (0.69)	read (0.60)



Connection to dimension reduction

- Embeddings are a form of **dimension reduction**: from a K-dimensional one-hot vector ($K \approx 100{,}000$) to d dimensions.
- The Austen embeddings on the last slide came from an **SVD** of a word-by-word count matrix, the same calculation as PCA. Early methods (latent semantic analysis) worked this way.
- LLM embeddings are instead learned to be useful for **prediction** (supervised by the next token), not to explain variance.
- We can use embeddings as **features** in the models we already know:
  - Embed each clinical note (one vector per note).
  - Fit a penalized logistic regression or random forest on the embedding.
  - Cluster the embeddings to find themes in free-text responses.

Connection to dimension reduction

- Embeddings are a form of **dimension reduction**: from a K-dimensional one-hot vector ($K \approx 100{,}000$) to d dimensions.
- The Austen embeddings on the last slide came from an **SVD** of a word-by-word count matrix, the same calculation as PCA. Early methods (latent semantic analysis) worked this way.
- LLM embeddings are instead learned to be useful for **prediction** (supervised by the next token), not to explain variance.
- We can use embeddings as **features** in the models we already know:
  - Embed each clinical note (one vector per note).
  - Fit a penalized logistic regression or random forest on the embedding.
  - Cluster the embeddings to find themes in free-text responses.

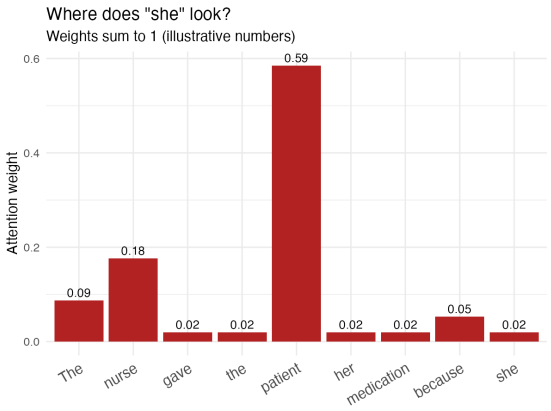
We will do exactly this in the NLP applications examples.

The problem with one vector per word

- In the embeddings so far, each word has **one** vector, no matter the context.
- But meaning depends on context:
  - "The patient has a **cold**." vs. "The patient's hands are **cold**."
  - "Patient tested **negative**." vs. "Patient has a **negative** attitude."
  - "**No** fever, **no** cough" — the word "no" changes the meaning of the words after it.
- We want **contextual embeddings**: the vector for "cold" should change depending on the words around it.
- This is what **attention** does.

Attention: the intuition

- To update a word's vector, decide **how much attention to pay** to each other word.
- "The nurse gave the patient her medication because **she** was in pain."
- To understand "she", the model should look mostly at "patient" (and a little at "nurse").
- The new vector for "she" is a **weighted average** of the vectors of the words it looks at.
- The weights differ for every sentence.



Attention: queries, keys, and values

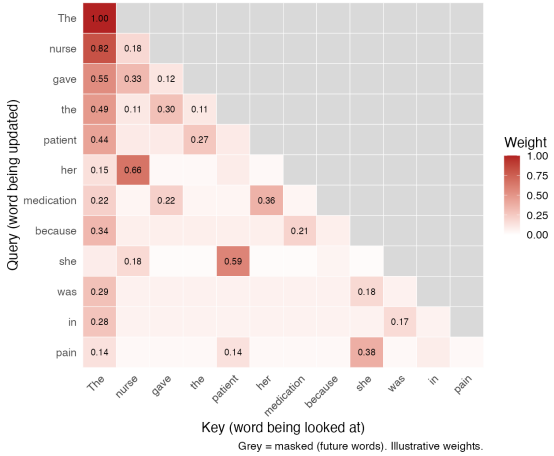
- Each token's vector $\boldsymbol{x}_i$ is turned into three vectors by multiplying by learned weight matrices:
  - **Query** $\boldsymbol{q}_i = \boldsymbol{W}_Q \boldsymbol{x}_i$: "what am I looking for?"
  - **Key** $\boldsymbol{k}_j = \boldsymbol{W}_K \boldsymbol{x}_j$: "what do I contain?"
  - **Value** $\boldsymbol{v}_j = \boldsymbol{W}_V \boldsymbol{x}_j$: "what information do I pass on?"
- Attention weight of token i on token j (a softmax!):

$$a_{ij} = \frac{\exp(\boldsymbol{q}_i^\top \boldsymbol{k}_j / \sqrt{d})}{\sum_l \exp(\boldsymbol{q}_i^\top \boldsymbol{k}_l / \sqrt{d})}.$$

- New vector for token i: $\boldsymbol{h}_i = \sum_j a_{ij} \boldsymbol{v}_j$.
- In matrix form: $\text{Attention}(\boldsymbol{Q}, \boldsymbol{K}, \boldsymbol{V}) = \text{softmax}\left(\boldsymbol{Q}\boldsymbol{K}^\top / \sqrt{d}\right) \boldsymbol{V}$.

Attention as a weighted average

- The weights a_{ij} form a matrix; each row sums to 1. The "she" row is the bar chart from two slides ago.
- Like a **kernel smoother** or K-nearest neighbors: each word's new value is a weighted average of the others, with weights from similarity ($\boldsymbol{q}_i^\top \boldsymbol{k}_j$).
- For text generation, token i can only look at tokens $j \leq i$ (**causal mask**, grey), so the model can't peek at the answer.

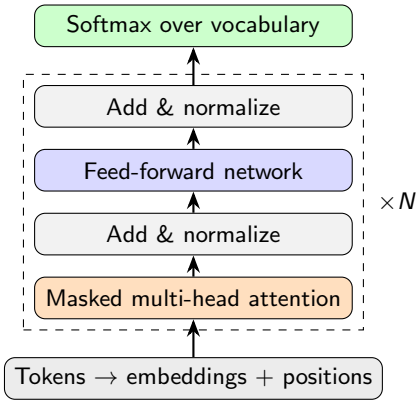


Multi-head attention and positions

- **Multi-head attention:** run several attention operations side by side, each with its own $\boldsymbol{W}_Q, \boldsymbol{W}_K, \boldsymbol{W}_V$, and combine the results.
  - Each head can learn a different kind of relationship: one looks at the previous word, one links pronouns to nouns, one tracks negation ("**no** evidence of pneumonia").
  - GPT-3 uses 96 heads in each layer.
- **Positional encoding:** attention by itself ignores word order, so "dog bites man" and "man bites dog" would look the same.
  - Fix: add a vector that encodes each token's position to its embedding.

The Transformer block

- One **Transformer block**:
  1. Multi-head attention (words share information).
  2. A feed-forward neural network applied to each word separately (like the networks we fit with `torch`).
  3. "Add & normalize" steps that help training.
- Stack many blocks: GPT-3 has 96.
- Most of the parameters are in the feed-forward layers.
- Everything is matrix multiplication, so it runs fast on GPUs.



Three kinds of Transformer models

Type	Examples	Good for
Encoder-only	BERT, BioBERT, Clinical-BERT	Classification, extraction, embeddings (sees words on both sides).
Decoder-only	GPT, Claude, Llama, Gemini	Generating text; chat; most modern LLMs.
Encoder–decoder	T5, original Transformer	Translation, summarization.

- BERT-type models are trained by **masking** words and predicting them ("fill in the blank").
- GPT-type models are trained by predicting the **next** word.
- For many public health classification tasks, a fine-tuned BERT-type model is cheaper and as accurate as a large chat model.

Step 1: Pretraining

- Predict the next token on huge amounts of text: web pages, books, code, scientific papers, Wikipedia.
- Trillions of tokens, billions of parameters, months on thousands of GPUs.
- Loss: cross-entropy, as in multinomial logistic regression.
- Result: a **base model** that is very good at continuing text, but not at following instructions.

Prompt to a base model	A likely continuation
What are the symptoms of measles?	What are the symptoms of mumps? What are the symptoms of rubella? ...

Step 1: Pretraining

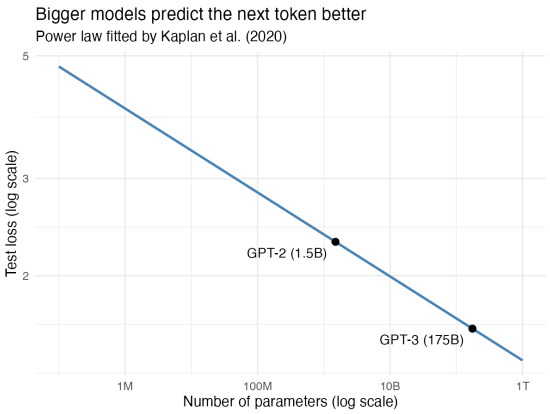
- Predict the next token on huge amounts of text: web pages, books, code, scientific papers, Wikipedia.
- Trillions of tokens, billions of parameters, months on thousands of GPUs.
- Loss: cross-entropy, as in multinomial logistic regression.
- Result: a **base model** that is very good at continuing text, but not at following instructions.

Prompt to a base model	A likely continuation
What are the symptoms of measles?	What are the symptoms of mumps? What are the symptoms of rubella? ...

Why? In web text, a question is often followed by *more questions* (e.g., an FAQ page).

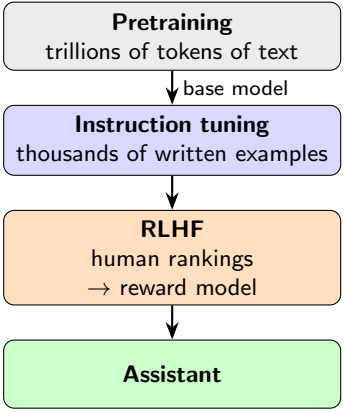
Scaling laws

- Test loss falls along a straight line on a log–log plot (a **power law**) as we increase
  - the number of parameters,
  - the amount of training data,
  - the compute.
- Hoffmann et al. (2022, "Chinchilla"): about 20 training tokens per parameter is compute-efficient.
- **Discussion:** why don't these huge models overfit, as in the bias–variance examples from early in the course?



Step 2: Instruction tuning and human feedback

- **Instruction tuning:** keep training on (instruction, good response) pairs written by people.
- **RL from human feedback (RLHF):**
  1. The model writes several answers to a prompt.
  2. People rank the answers.
  3. Fit a **reward model** to predict the rankings (a pairwise logistic regression!).
  4. Tune the LLM to produce answers with high reward.
- Result: an **assistant** that answers the measles question instead of continuing it.



Fine-tuning for your own task

- You can also fine-tune a pretrained model on your own labeled data, e.g., 2,000 notes labeled "opioid overdose: yes/no".
- **Transfer learning:** the pretrained model already "knows" language; fine-tuning only teaches the task. Far less labeled data is needed than training from scratch.
- Common approaches:
  - Add a classification layer on top of a BERT-type model and train all weights.
  - **Parameter-efficient fine-tuning** (e.g., LoRA): freeze the model and train a small number of extra parameters.
- All of the usual rules apply: train/validation/test splits, cross-validation, class imbalance, calibration, PR-AUC.

How an LLM generates text

1. Feed the text so far through the model.
2. Get the next-token probabilities.
3. **Sample** a token from that distribution.
4. Append it to the text and go back to step 1.

- One token at a time; no "plan" unless it writes one out.
- Because of sampling, the same prompt can give different answers.

Text so far	Next token
Common symptoms of influenza are	

How an LLM generates text

1. Feed the text so far through the model.
2. Get the next-token probabilities.
3. **Sample** a token from that distribution.
4. Append it to the text and go back to step 1.

- One token at a time; no "plan" unless it writes one out.
- Because of sampling, the same prompt can give different answers.

Text so far	Next token
Common symptoms of influenza are	**fever**

How an LLM generates text

1. Feed the text so far through the model.
2. Get the next-token probabilities.
3. **Sample** a token from that distribution.
4. Append it to the text and go back to step 1.

- One token at a time; no "plan" unless it writes one out.
- Because of sampling, the same prompt can give different answers.

Text so far	Next token
Common symptoms of influenza are	**fever**
. . . influenza are fever	**,**

How an LLM generates text

1. Feed the text so far through the model.
2. Get the next-token probabilities.
3. **Sample** a token from that distribution.
4. Append it to the text and go back to step 1.

- One token at a time; no "plan" unless it writes one out.
- Because of sampling, the same prompt can give different answers.

Text so far	Next token
Common symptoms of influenza are	**fever**
. . . influenza are fever	**,**
. . . influenza are fever,	**cough**

How an LLM generates text

1. Feed the text so far through the model.
2. Get the next-token probabilities.
3. **Sample** a token from that distribution.
4. Append it to the text and go back to step 1.

- One token at a time; no "plan" unless it writes one out.
- Because of sampling, the same prompt can give different answers.

Text so far	Next token
Common symptoms of influenza are	**fever**
. . . influenza are fever	**,**
. . . influenza are fever,	**cough**
. . . fever, cough	**and**

How an LLM generates text

1. Feed the text so far through the model.
2. Get the next-token probabilities.
3. **Sample** a token from that distribution.
4. Append it to the text and go back to step 1.

- One token at a time; no "plan" unless it writes one out.
- Because of sampling, the same prompt can give different answers.

Text so far	Next token
Common symptoms of influenza are	**fever**
. . . influenza are fever	**,**
. . . influenza are fever,	**cough**
. . . fever, cough	**and**
. . . fever, cough and	**body**

How an LLM generates text

1. Feed the text so far through the model.
2. Get the next-token probabilities.
3. **Sample** a token from that distribution.
4. Append it to the text and go back to step 1.

- One token at a time; no "plan" unless it writes one out.
- Because of sampling, the same prompt can give different answers.

Text so far	Next token
Common symptoms of influenza are	**fever**
. . . influenza are fever	**,**
. . . influenza are fever,	**cough**
. . . fever, cough	**and**
. . . fever, cough and	**body**
. . . cough and body	**aches**

Each row is one pass through the whole network.

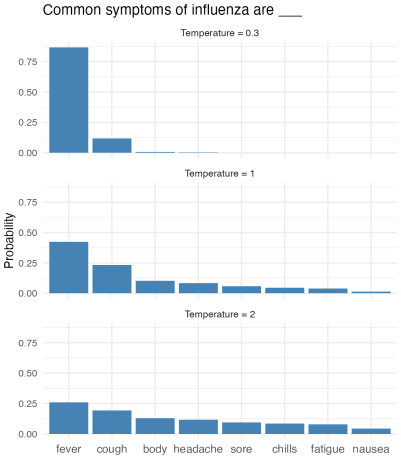
Temperature and sampling

- **Temperature** τ rescales the softmax:

$$p_k(\tau) = \frac{\exp(z_k/\tau)}{\sum_j \exp(z_j/\tau)}.$$

  - $\tau \to 0$: always the most likely token (greedy).
  - $\tau = 1$: the model's distribution.
  - $\tau > 1$: flatter, more random.

- **Top-p sampling:** sample only from the most likely tokens whose probabilities sum to p.
- For research use, set a low temperature and report it.



Prompting

- **Zero-shot:** just ask.
- **Few-shot:** include a few labeled examples in the prompt. No weights change!
- **Chain-of-thought:** ask for step-by-step reasoning first. "Reasoning" models do this automatically.
- **Structured output:** ask for JavaScript Object Notation (JSON) or fixed labels to read into R .
- The prompt is a tuning parameter: develop it on a **validation set**; evaluate once on a **test set**.

A few-shot prompt

```
Classify each ED triage note as fall-related
or not. Answer in JSON.

Note: "Pt tripped on stairs, L wrist pain"
Answer: {"fall": "yes"}

Note: "Chest pain x2 hrs, diaphoretic"
Answer: {"fall": "no"}

Note: "Slipped on ice outside home, hit head"
Answer:
```

Model output: `{"fall": "yes"}`

Context windows

- The **context window** is the maximum number of tokens the model can see at once (prompt + answer).
- It has grown from 512 tokens (BERT) to hundreds of thousands or millions of tokens.
- Attention compares every token to every other token, so cost grows like $(\text{length})^2$.
- Longer is not always better: models can miss information in the middle of a long context ("lost in the middle").
- The model has no memory between conversations except what is put into the context.

Context windows

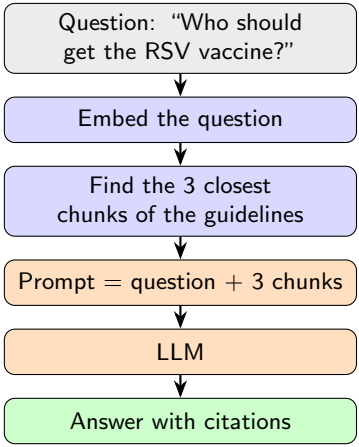
- The **context window** is the maximum number of tokens the model can see at once (prompt + answer).
- It has grown from 512 tokens (BERT) to hundreds of thousands or millions of tokens.
- Attention compares every token to every other token, so cost grows like $(\text{length})^2$.
- Longer is not always better: models can miss information in the middle of a long context ("lost in the middle").
- The model has no memory between conversations except what is put into the context.

Question: A patient's full EHR is 2 million tokens. How would you get an LLM to answer "Has this patient ever had a stroke?"

Retrieval-augmented generation (RAG)

1. Split documents (guidelines, papers) into chunks; embed each chunk.
2. Embed the question; find the most similar chunks (nearest neighbors by cosine similarity).
3. Put those chunks in the prompt; ask for an answer with citations.

- Fewer made-up answers; can use current or private information.
- Only as good as the retrieval step.
- One answer to the EHR question on the last slide.



Tools and agents

- Modern LLMs can call **tools**: run code, search the web, query a database, use a calculator.
- An **agent** is an LLM in a loop: it plans, calls tools, looks at the results, and continues until the task is done.
- Examples: an assistant that writes and runs `R` code, fixes errors, and makes a plot; a coding assistant that edits files in a project.
- Benefits: fewer arithmetic mistakes; can work with real data.
- Risks: errors can compound over many steps; you must check the work (and the code).

Hallucinations

- LLMs are trained to produce **plausible** text, not **true** text.
- **Hallucination:** fluent, confident, and wrong. Common examples:
  - made-up references (real authors and journals, papers that don't exist),
  - wrong drug doses,
  - invented statistics.
- Why? Next-token prediction rewards a confident-sounding continuation, and "I don't know" is rare in training text.
- Mitigations: RAG with citations, asking for evidence, low temperature, and **always** human checking for important uses.

Hallucinations

- LLMs are trained to produce **plausible** text, not **true** text.
- **Hallucination:** fluent, confident, and wrong. Common examples:
  - made-up references (real authors and journals, papers that don't exist),
  - wrong drug doses,
  - invented statistics.
- Why? Next-token prediction rewards a confident-sounding continuation, and "I don't know" is rare in training text.
- Mitigations: RAG with citations, asking for evidence, low temperature, and **always** human checking for important uses.

Try it: ask a chatbot for five references on a narrow topic you know well, then look them up.

Bias and fairness

- LLMs learn from human-written text, including its biases.
- Documented examples in health:
  - Repeating race-based medical myths (e.g., race-based kidney function or lung capacity equations).
  - Different recommendations when only the patient's race or gender in a vignette is changed.
  - Worse performance in languages and dialects that are less common in the training data.
- Instruction tuning and RLHF reduce some biases but can also add the preferences of the raters.
- Evaluate performance **by subgroup**, as we discussed for prediction models (more in the fairness lecture).

Privacy, reproducibility, and other concerns

- **Privacy:** never paste protected health information (PHI) into a public chatbot. Use approved, HIPAA-compliant deployments or locally run open-weight models. Check with your IRB.
- **Reproducibility:** commercial models are updated and retired; results may change. Report the model name and version, date, prompt, and settings (temperature).
- **Data contamination:** a model may have seen the test questions (e.g., medical board exams) during pretraining, inflating benchmark scores.
- **Calibration:** the model's stated confidence ("I'm 90% sure") is often not a calibrated probability.
- **Cost and environment:** large models use a lot of energy; a smaller fine-tuned model may do the job.

How are LLMs evaluated?

- **Benchmarks:** multiple-choice exams (e.g., MedQA, USMLE-style questions), coding tests, reasoning puzzles.
- Med-PaLM (2022) was the first model to pass the USMLE-style threshold; newer models score much higher.
- But passing an exam $\neq$ being a good clinician:
  - real patients are messy and their information is incomplete,
  - exam questions may be in the training data (contamination).
- **Human evaluation:** experts rate answers for accuracy, completeness, and possible harm.
- Benchmarks tell you about the model in general, not about **your** data and **your** task.

Evaluating an LLM for your study

When you use an LLM as a measurement or classification tool, evaluate it like any other classifier:

- Have experts label a **random sample** (the gold standard).
- Split into a development set (for the prompt) and a test set (used once).
- Report sensitivity, specificity, PPV, F1, and PR-AUC with confidence intervals.
- Compare against a simple baseline (keyword search, penalized logistic regression on word counts, a fine-tuned BERT model).
- Check performance by subgroup and by data source.
- Check stability: re-run the same prompts and look at agreement.

Evaluating an LLM for your study

When you use an LLM as a measurement or classification tool, evaluate it like any other classifier:

- Have experts label a **random sample** (the gold standard).
- Split into a development set (for the prompt) and a test set (used once).
- Report sensitivity, specificity, PPV, F1, and PR-AUC with confidence intervals.
- Compare against a simple baseline (keyword search, penalized logistic regression on word counts, a fine-tuned BERT model).
- Check performance by subgroup and by data source.
- Check stability: re-run the same prompts and look at agreement.

An LLM is a measurement instrument. Its errors are measurement error, and they can bias downstream analyses.

Public health applications (next lecture)

- Classifying clinical notes (e.g., overdose, falls, suicide risk).
- Extracting structured data from free text (smoking status, symptoms, social determinants of health).
- Syndromic surveillance from ED chief complaints and social media.
- Coding causes of death and verbal autopsies.
- Screening literature for systematic reviews.
- Open-ended survey responses (topics, sentiment).

Discussion: which of these could you do with the methods we already know (word counts + penalized regression, clustering)? Where would an LLM help?

Summary

- An LLM is a neural network trained for one classification task: **predict the next token**, with a softmax and cross-entropy loss.
- Text is split into **tokens**; tokens become **embedding** vectors.
- **Attention** lets each token's vector depend on its context; stacked **Transformer** blocks do this repeatedly.
- Pretraining on huge data gives a base model; **instruction tuning and RLHF** make it an assistant.
- Use LLMs with prompting, RAG, fine-tuning, or embeddings as features.
- Be aware of hallucination, bias, privacy, and reproducibility, and **evaluate them like any other model**.

References

- Vaswani A, et al. (2017). Attention is all you need. *NeurIPS*.
- Mikolov T, et al. (2013). Efficient estimation of word representations in vector space. *arXiv:1301.3781*.
- Devlin J, et al. (2019). BERT: Pre-training of deep bidirectional transformers for language understanding. *NAACL*.
- Brown T, et al. (2020). Language models are few-shot learners. *NeurIPS*.
- Kaplan J, et al. (2020). Scaling laws for neural language models. *arXiv:2001.08361*.
- Ouyang L, et al. (2022). Training language models to follow instructions with human feedback. *NeurIPS*.
- Singhal K, et al. (2023). Large language models encode clinical knowledge. *Nature*.
- Jurafsky D, Martin JH. *Speech and Language Processing*, 3rd ed. web.stanford.edu/∼jurafsky/slp3
- Alammar J. The Illustrated Transformer. jalammar.github.io